

C5W4

Transformer Network Intuition

⇒ Transformer does 2 things brilliantly

→ Attention mechanism

→ Parallel computation of features like CNN.

→ So the attention mechanism has 2 types

(1) Self attention

(2) Multi head attention.

Self Attention

→ For intuition just watch ^{sep} 3blue1brown's ~chap deep learning chapter 6, & i have the intuition now but idk it might go away in future & id want to write the whole intuition here, besides it would be much easier for you because i understand it RN.

Now let's continue with the technical jargon shall we?

→ To use attention in the style of CNN you need self

→ $A(q, k, v)$ = attention-based vector representation of a word.

→ This "A" vector is calculated for each word.

$A^{<1>} \dots A^{<N>}$

→ & these vector representation calculations are done in parallel.

RNN Attention

$$a^{<t, t'>} = \frac{\exp(e^{<t, t'>})}{\sum_{t'=1}^{T_n} \exp(e^{<t, t'>})}$$

Transformer Attention $v^{<i>}$

$$A(q, k, v) = \sum_i \frac{\exp(q \cdot k^{<i>})}{\sum_j \exp(q \cdot k^{<j>})} v^{<i>}$$

Here, the equations of Attention for RNN's & Transformers RNN's look quite similar visually.

Here, in the Transformers,

we will be using
 $q^{<i>}$ → query
 $k^{<i>}$ → key
 $v^{<i>}$ → values
 These are assigned for each token in the sentence to compute the vector representation.

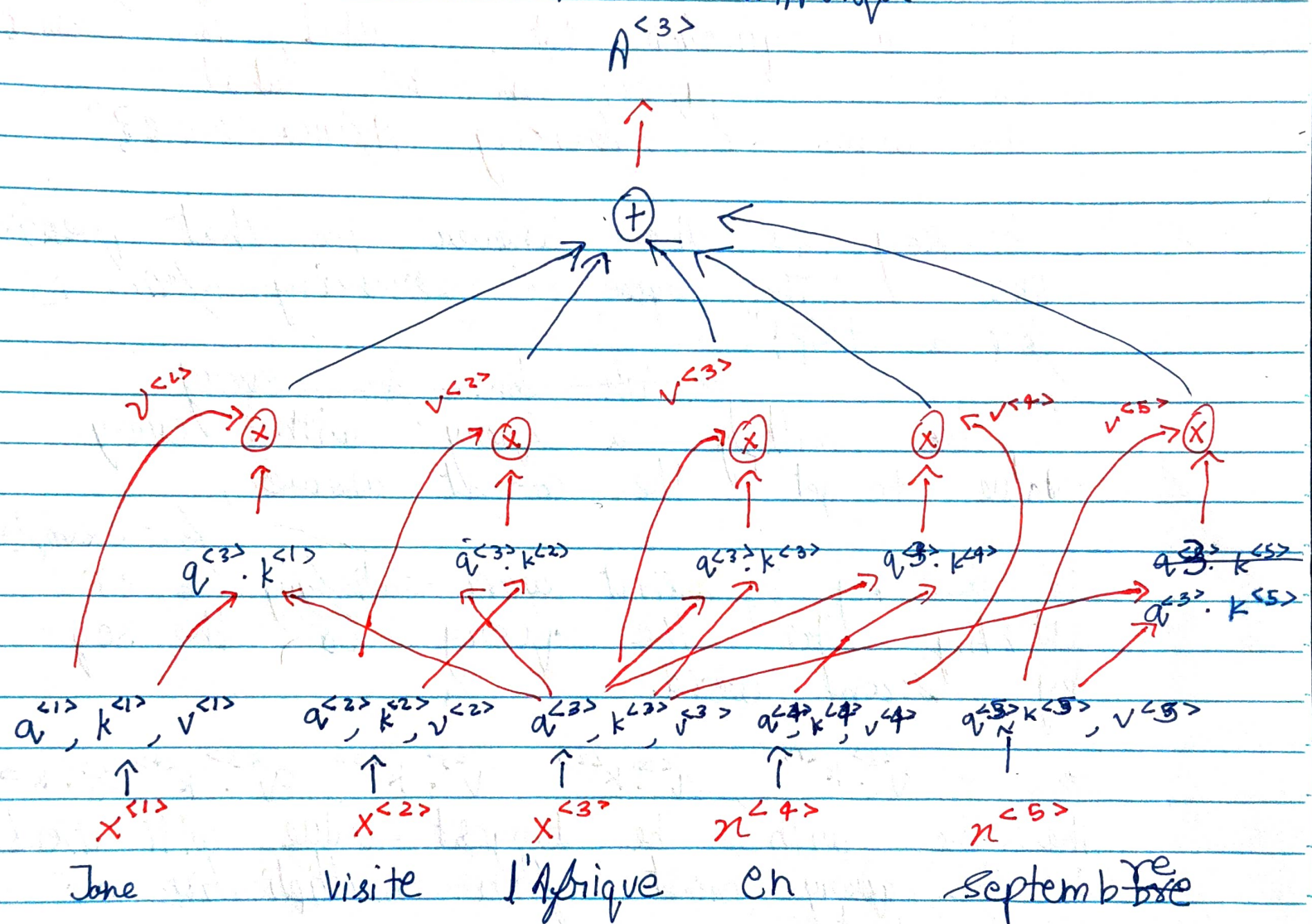
representation.

for example, we will take the sentence

$x^{<1>}$ $x^{<2>}$ $x^{<3>}$ $x^{<4>}$ $x^{<5>}$
 Jone visite l'Afrique en Septembre

Self Attention ("Real source")

Let's use the word $x^{(3)}$ to determine the process which is "l'Afrique" to get $A^{(3)}$, the word is l'Afrique



→ first we will be associating each of the words with query "q", key "k" & value "v".

→ taking word embedding $x^{(3)}$, $q^{(3)} = W^q \cdot x^{(3)}$
 $k^{(3)} = W^k \cdot x^{(3)}$
 $v^{(3)} = W^v \cdot x^{(3)}$

above, w^Q , w^K & w^V are the learnable parameters, & $x^{<i>3>}$ is the word embeddings

So,

their loose analogies are that, query is a question that is asked, for example in $x^{<3>}$, query can be, "what is the purpose of referencing Africa here?"

& key is the answer to that question like "The purpose of referencing Africa is for a trip!"

& it's done for every & we multiply a query with every value to get the correct answer,

for example is the key by first word satisfying or attending to the query or the key by second word.

$\Rightarrow q^{<3>} \cdot k^{<1>}, q^{<3>} \cdot k^{<2>}, q^{<3>} \cdot k^{<3>}, q^{<3>} \cdot k^{<4>}, q^{<3>} \cdot k^{<5>}$
the one with the largest value will attend to the query most after multiplication or dot product,

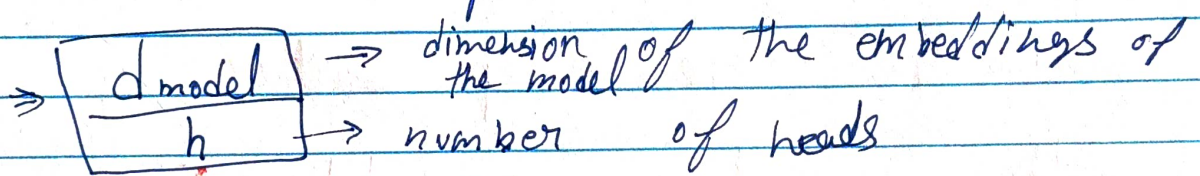
& we will compute the softmax over all the values.

to get a number from 0 to 1, & finally

MULTI-HEAD

Attention

In the multi head attention, we have multiple self attention blocks outputting the vector representations & each ~~the~~ self attention output has the dimension of,



So, if the dimension of the model is 128 & the number of heads are 8

then every head or every self attention block will output a vector of 16 dimensions,

So, after taking ~~the~~ the output of every self attention block or head, we concatenate it

like, (head 1, head 2, head 3, - - - -)

then, we do multiply this concatenated representation ~~vector~~ to another matrix

$\rightarrow$ weights
 "Wo", ~~to~~ to get the final vector representation
 & then we add this new vector representation
 to the original embedding representation.
 to get the final representation.

$$\Rightarrow \text{head}_i = \text{Attention}(w_i^Q Q, w_i^K K, w_i^V V)$$

$$\Delta \vec{E} \Rightarrow \text{MultiHead}(Q, K, V) = \text{concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) W_O$$

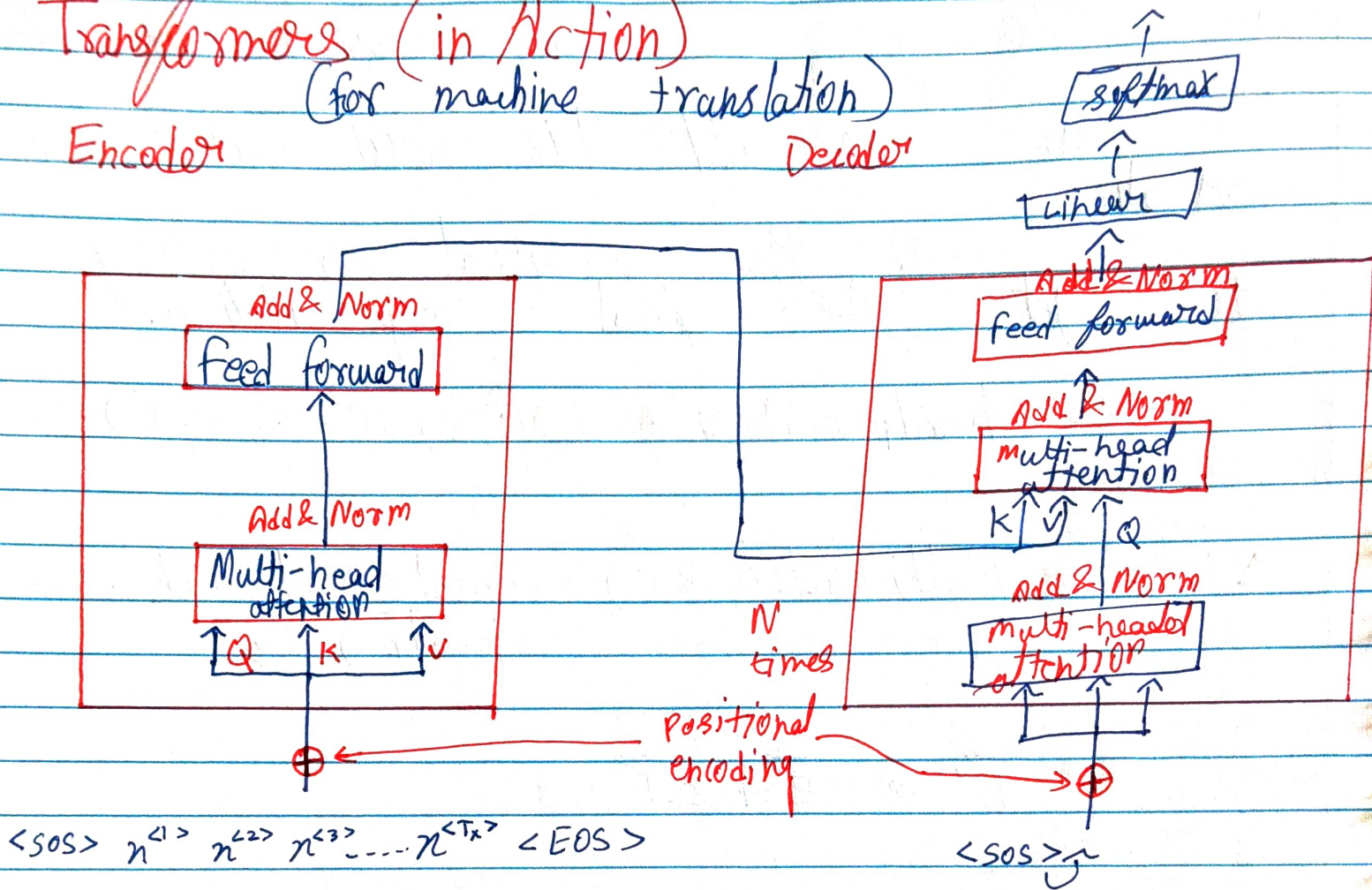
$$\vec{E}' \Rightarrow \vec{E} + \Delta \vec{E}$$

Transformers (in Action)

(for machine translation)

Encoder

Decoder



→ So,

what we have here is that in machine translation, type architecture of the transformer, we have encoder & decoders

→ So encoder takes all the tokens as input in which is in different language & we get "K & V" values from them

Now, on the decoder side

→ It uses cross attention (YK) "kV" from encoder
& Q from decoder

we input the $\langle \text{eos} \rangle$ (start of sentence) token
& then to the first, multi head attention block
& after that,

it generates queries for the
input token (which in this case is $\langle \text{eos} \rangle$) &

after that you do multihead attention
computation,

but only to take "k & V" from
encoder & "Q" from decoder &

feed the output to the "MLP" then
layer & then just,

"woooo hoooo!!" linear & softmax
tokens, you get the output

how you join that output token
from the decoder to the input token
& do the same shit till you get
 $\langle \text{Eos} \rangle$

tokens or just stop after the "N" number
of times,

→ In transformers

→ In transformers for LLM's we use masked multi head attention while training, all it does is mask out the last ~~sentence~~ word & forces model to predict it.

Transformers (for LLM's)

→ So yk,

encoder & whatever you're left with is ~~that~~ an

transformer for LLM Lmal.
→ Just ~~not~~ use the last token representation for the next token
→ & Yeah gges,

but let's just cover some topics for sampling  { I didn't intend to create rick

Fk i forgot to talk about positional encoding maybe i should stop goofing around

Positional encoding

→ In the original Transformer they used Sinusoidal positional embedding,

where they, ex → embedding token

~~←~~ ~~→~~ ~~↔~~

$x^{(t)}$ is 4 dimensional

$\begin{bmatrix} \end{bmatrix}_{4 \times 1}$

so, position vector will also be 4 dimensional

$\begin{bmatrix} \end{bmatrix}_{4 \times 1}$
p < 12

Now, using the formula

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{(10000)^{2i/d}}\right)$$

$$\& PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{(10k)^{2i/d}}\right)$$

$$\cos\left(\frac{pos}{(10k)^{2i/d}}\right)$$

Here "i" is the tells us the which pair of dimension we are computing

for example let's say the dimension of embedding is '10' then positional embedding dimension is also 10

& the pairs will look like

index	0	1	2	3	4	5	6	7	8	9
"i" value	0	0	1	1	2	2	3	3	4	4

& Then finally, the final vector will look like

$$x^{<1>} = \begin{bmatrix} \end{bmatrix}_{4 \times 1}^{x^{<1>}} + \begin{bmatrix} \end{bmatrix}_{4 \times 1}^{p^{<1>}}$$

But in the modern LLM architecture we use

"ROPE" $\rightarrow$ Rotary positional embeddings.

$\rightarrow$ In this method of positional embedding we rotate the ~~the~~ embedding of the vector to a certain angle, but "rope" don't rotate the entire vector it rotates the pairs creates pairs & then rotate the pairs by certain angle.

ex $\rightarrow$ [1, 3, 6, 8, 12, 15]

all of the above pairs get rotated
(1, 3), (6, 8), (12, 15)

$\rightarrow$ we only apply ROPE to "Q" & "K" not "V"

$\rightarrow$ it is applied multiple times in the one transformer block

$\rightarrow$ It is applied one per transformer block

& here is a diagram where it fits in the pipeline

A single transformer block

Input Embedding



Enter transformer block



Layer norm (Pre-norm architecture)



Multi head attention

→ $Q = X W_Q$

$K = X W_K$

$V = X W_V$

→ [RoPE is applied here]

→ apply mathematical rotation to Q & K matrix to get $\tilde{Q}$ & $\tilde{K}$

→ No rotation on V

→ Calculate softmax $\left(\frac{\tilde{Q} \cdot \tilde{K}^T}{\sqrt{d}} \right)$

→ Multiply by values : output = Score $\times V$

→ Project through final output weights (W_O)



Add & Norm (Residual connection from before attention)



MLP / Feed forward Network



Add & Norm (Residual connection from before MLP)

Sampling

The modern LLMs use a mixture of sampling techniques like,

- i) Temperature
- ii) Top-k Sampling
- iii) Top-P (Nucleus sampling)

i) Temperature

$$P_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

here we introduce a new hyperparameter "T"

if "T" is greater than 1 then it starts to flatten out the probabilities & makes the model less confident & creative

if "T" is ($0 < T < 1$) Then it makes the model more confident on a certain ~~or~~ or highlights a specific probability output

ii) Top-k Sampling :- Here we say that we will be sampling only from top "K" number of probabilities,

for example if $K=3$, then

→ Hey	93%
→ i	2.5%
→ am	1%
→ I'm a	0.5%

→ we will only sample from these

iii) Top-P (Nucleus Sampling)

→ here we say keep enough tokens to account for P% of total probability mass

example ,

red	30%
blue	25%
green	20%
yellow	15%
orange	10%

for $P = 0.9$ we only keep,

red	30
blue	25
green	20
yellow	15
$\Rightarrow 90\%$	

& discard other values.

Here is a code snippet example -:

```
model.generate (
    temperature = 0.7,
    top-p top-p = 0.9,
    top-k = 50 )
```